Intro to Data Science - Lab 5

Copyright 2025, Jeffrey Stanton and Jeffrey Saltz Please do not post online.

Week 5 - Obtaining and Using Data from a JSON API

```
# Enter your name here: Tim Soron
```

Please include nice comments.

Instructions:

Run the necessary code on your own instance of R-Studio.

Attribution statement: (choose only one and delete the rest)

```
# 1. I did this lab assignment by myself, with help from the book and the professor.
```

JSON (JavaScript Object Notation) provides a transparent, user friendly data exchange format that many organizations use to make their data available over the web. JSON is human readable, but is also highly structured, and its support for nested data structures makes it highly flexible. Today we will use JSON to obtain data from the **NYC CitiBike project**. The CitiBike project provides an application programming interface (API) that members of the public can access to get up-todate information on the status of more than 800 bike stations. You may need to install the **RCurl** and **jsonlite** packages to get the code to work.

```
#install.packages('RCurl')
#install.packages('jsonlite')
```

```
station_link <- 'https://gbfs.citibikenyc.com/gbfs/en/station_status.json'
apiOutput <- getURL(station_link)
apiData <- fromJSON(apiOutput)
stationStatus <- apiData$data$stations
cols <- c('num_bikes_disabled','num_docks_disabled', 'station_id',
 'num_ebikes_available', 'num_bikes_available', 'num_docks_available')
stationStatus = stationStatus[,cols]
```

1. Explain what you see if you type in the **station_link** URL into a browser (in a comment, write what you see)

```
# It looks very complicated and hard to understand
# Objects are enclosed in {} and contain key value pairs
# Arrays are enclosed in [] and contain values
# Values can be strings, numbers, boolean, null, or nested object or arrays
```

2. Paste the code from above here and provide a comment explaining each line of code.

```
station_link <- 'https://gbfs.citibikenyc.com/gbfs/en/station_status.json'
# Generated the variable with text that represents the url

library(RCurl)
apiOutput <- getURL(station_link)
# The API output downloaded the json file as one large text string

library(jsonlite)
apiData <- fromJSON(apiOutput)
#Parse json string into a R list
```

```r
stationStatus <- apiData$data$stations
# Extract the 'Stations' dataframe nested in the parsed json object

cols <- c('num_bikes_disabled','num_docks_disabled', 'station_id',
 'num_ebikes_available', 'num_bikes_available', 'num_docks_available')
stationStatus = stationStatus[,cols]
```

3. Use **str( )** to find out the structure of **apiOutput** and **apiData**. Report (via a comment) what you found and explain the difference between these two objects.

```r
str(apiOutput)
```

```
##  chr "{\"data\": {\"stations\": [{\"is_renting\": 0, \"num_bikes_disabled\": 0, \"num_bikes_availabl
```

```r
str(apiData)
```

```
## List of 4
##  $ data        :List of 1
##   ..$ stations:'data.frame': 2354 obs. of  14 variables:
##   .. ..$ is_renting               : int [1:2354] 0 0 0 0 0 0 0 0 0 1 ...
##   .. ..$ num_bikes_disabled       : int [1:2354] 0 0 0 0 0 0 0 0 23 11 ...
##   .. ..$ num_bikes_available      : int [1:2354] 0 0 0 0 0 0 0 0 0 42 ...
##   .. ..$ num_docks_disabled       : int [1:2354] 0 0 0 0 0 0 0 0 0 0 ...
##   .. ..$ eightd_has_available_keys: logi [1:2354] FALSE FALSE FALSE FALSE FALSE FALSE ...
##   .. ..$ legacy_id                : chr [1:2354] "1905837242740508940" "4774" "2171904223419888148" 
##   .. ..$ is_installed             : int [1:2354] 0 0 0 0 0 0 0 0 1 1 ...
##   .. ..$ is_returning             : int [1:2354] 0 0 0 0 0 0 0 0 0 1 ...
##   .. ..$ num_docks_available      : int [1:2354] 0 0 0 0 0 0 0 0 2 1 ...
##   .. ..$ last_reported            : int [1:2354] 86400 86400 86400 86400 86400 86400 86400 1771339288
##   .. ..$ num_ebikes_available     : int [1:2354] 0 0 0 0 0 0 0 0 0 5 ...
##   .. ..$ station_id               : chr [1:2354] "1905837242740508940" "41495491-5d89-4e14-aab9-c3db
##   .. ..$ num_scooters_unavailable : int [1:2354] NA NA NA NA NA NA NA NA 0 0 ...
##   .. ..$ num_scooters_available   : int [1:2354] NA NA NA NA NA NA NA NA 0 0 ...
##  $ last_updated: int 1771346856
##  $ ttl         : int 60
##  $ version     : chr "1.1"
```

```r
# The apiOutput returns a character while the apiData returns a list of 4 and is more structured
```

4. The **apiOutput** object can also be examined with a custom function from the **jsonlite** package called **prettify( )**. Run this command and explain what you found (in a comment).

```r
#prettify(apiOutput)

#it returns all of the data inside
```

5. Explain **stationStatus** (what type of object, what information is available).

```r
str(stationStatus)
```

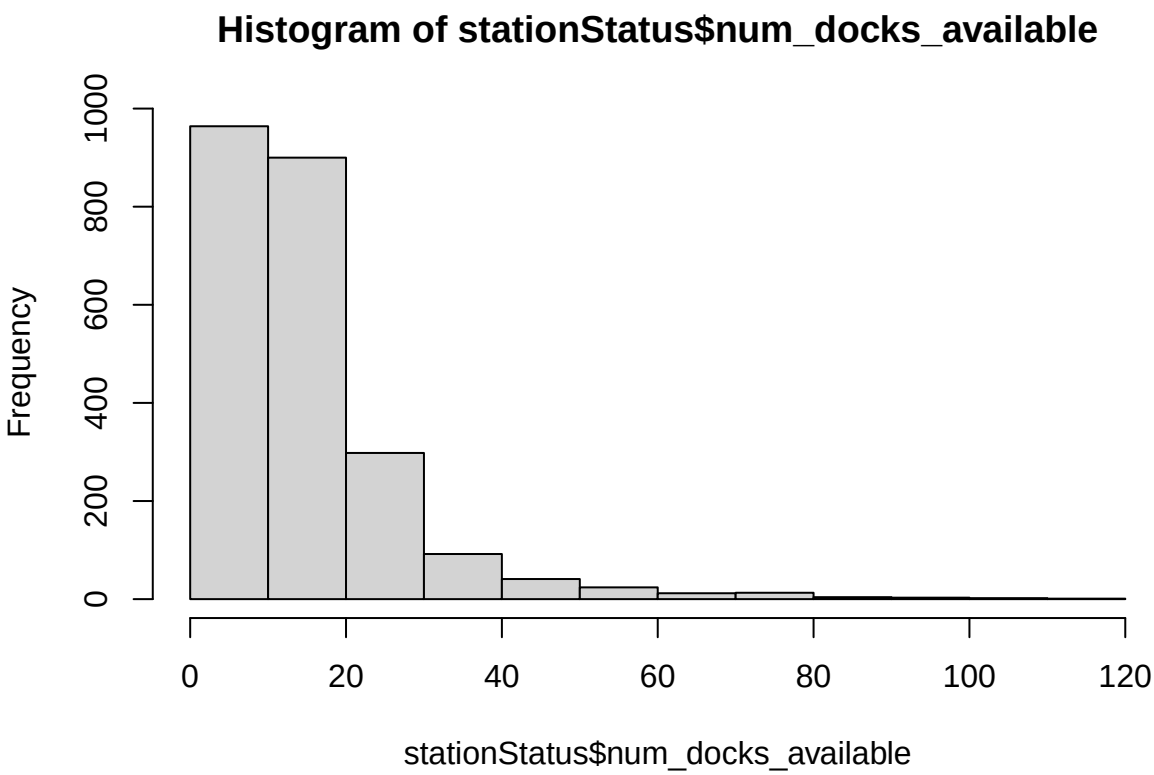
```
## 'data.frame':    2354 obs. of  6 variables:
##  $ num_bikes_disabled  : int  0 0 0 0 0 0 0 0 23 11 ...
##  $ num_docks_disabled  : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ station_id          : chr  "1905837242740508940" "41495491-5d89-4e14-aab9-c3db04aad399" "21719042
##  $ num_ebikes_available: int  0 0 0 0 0 0 0 0 0 5 ...
##  $ num_bikes_available : int  0 0 0 0 0 0 0 0 0 42 ...
##  $ num_docks_available : int  0 0 0 0 0 0 0 0 2 1 ...
```

```r
summary(stationStatus)
```

```
##  num_bikes_disabled num_docks_disabled  station_id        num_ebikes_available
##  Min.   : 0.000     Min.   : 0.0000    Length:2354        Min.   : 0.000
##  1st Qu.: 1.000     1st Qu.: 0.0000    Class :character   1st Qu.: 0.000
##  Median : 2.000     Median : 0.0000    Mode  :character   Median : 2.000
##  Mean   : 2.488     Mean   : 0.1232                       Mean   : 5.605
##  3rd Qu.: 3.000     3rd Qu.: 0.0000                       3rd Qu.: 5.000
##  Max.   :49.000     Max.   :41.0000                       Max.   :85.000
##  num_bikes_available num_docks_available
##  Min.   :  0.00      Min.   :  0.00
##  1st Qu.:  2.00      1st Qu.:  5.00
##  Median :  7.00      Median : 13.00
##  Mean   : 12.62      Mean   : 14.36
##  3rd Qu.: 17.00      3rd Qu.: 19.00
##  Max.   :115.00      Max.   :113.00
```

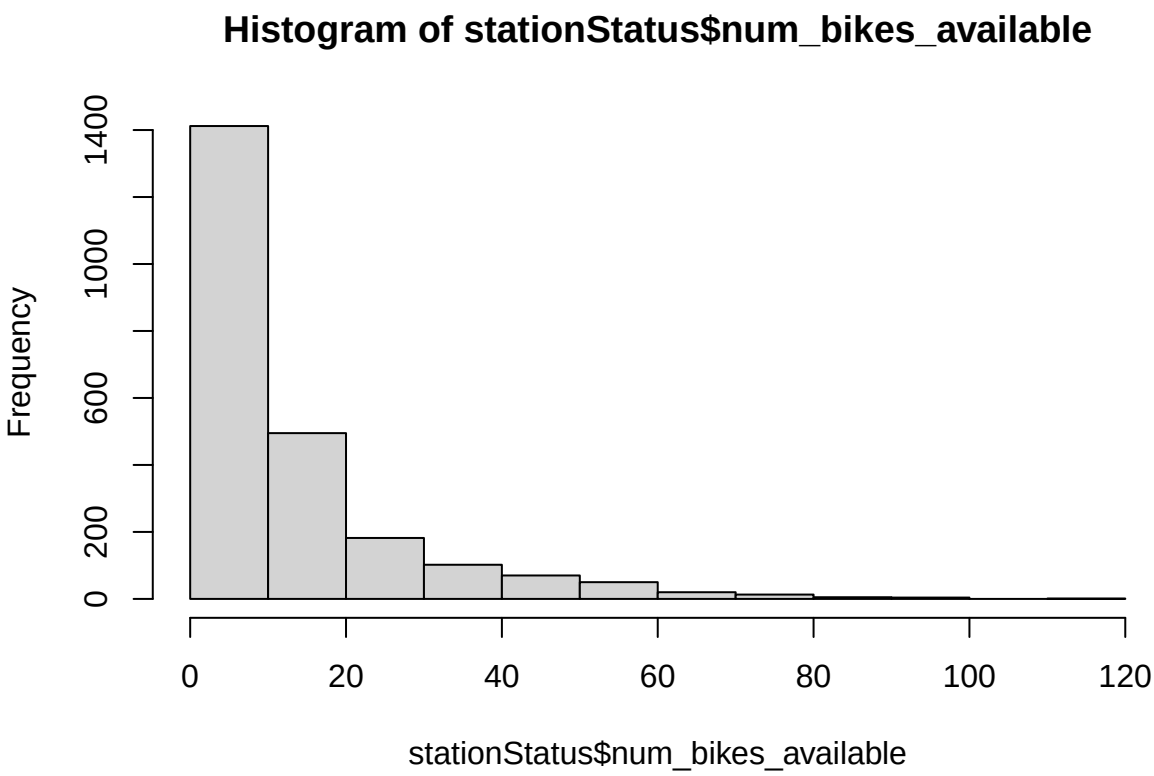
6. Generate a histogram of the number of docks available

```r
hist(stationStatus$num_docks_available)
```



7. Generate a histogram of the number of bikes available

```r
hist(stationStatus$num_bikes_available)
```



8. How many stations have at least one ebike?

```
sum(stationStatus$num_ebikes_available >= 1)
```

```
## [1] 1590
```

9. Explore stations with at least one ebike by create a new dataframe, that only has stations with at least one eBike.

```
ebike_stations <- stationStatus[stationStatus$num_ebikes_available>=1,]
```

10. Calculate the mean of ** num_docks_available ** for this new dataframe.

```
mean(ebike_stations$num_docks_available)
```

```
## [1] 12.79308
```

11. Calculate the mean of ** num_docks_available ** for for the full ** stationStatus ** dataframe. In a comment, explain how different are the two means?

```
mean(stationStatus$num_docks_available)
```

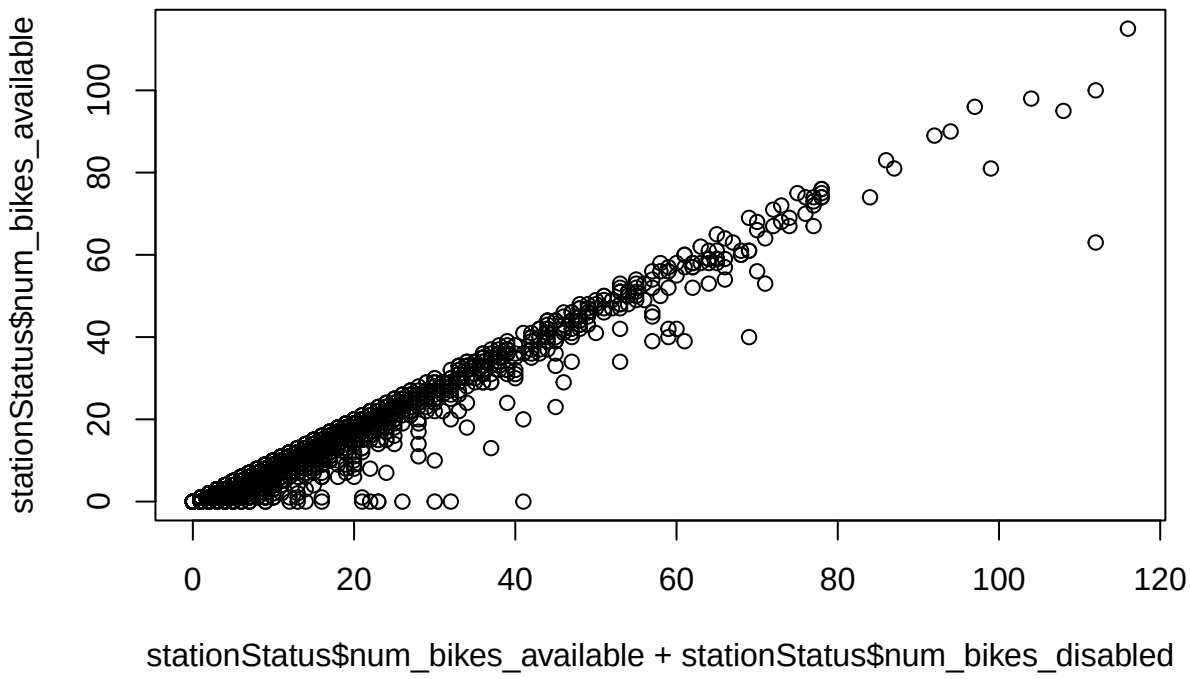
```
## [1] 14.35599
```

12. Create a new attribute, called ** stationSize **, which is the total number of slots available for a bike (that might, or might not, have a bike in it now). Run a histogram on this variable and review the distribution.

```
stationStatus$station_size <- stationStatus$num_docks_available + stationStatus$num_bikes_available + st
```

13. Use the **plot( )** command to produce an X-Y scatter plot with the number of occupied docks on the X-axis and the number of available bikes on the Y-axis. Explain the results plot.

```
plot(stationStatus$num_bikes_available + stationStatus$num_bikes_disabled, stationStatus$num_bikes_avai
```




```
# positve relationship between x and y variable, makes sense as they are apart of each other
```